

静态树分治与计算几何中的扫描线问题

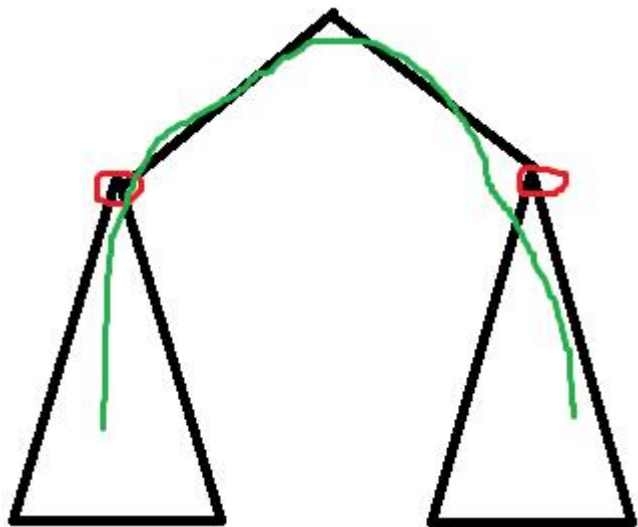
——成都七中 nzhtl1477

Loj 6276

- 树, 有多少链满足上面的颜色互不相同
- 颜色数20

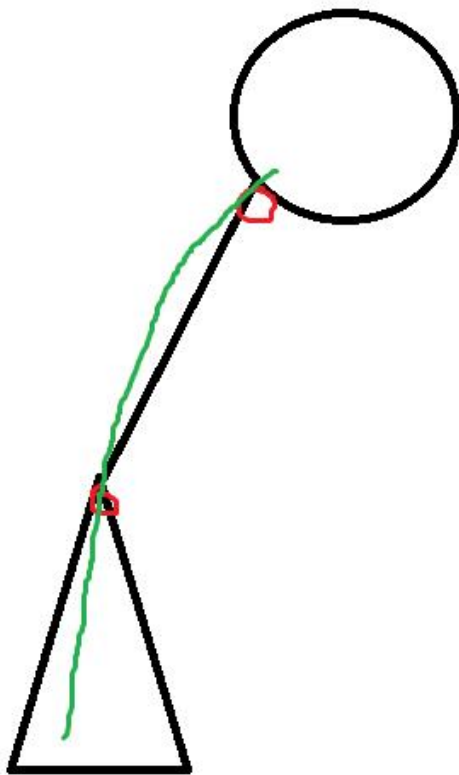
Solution

- 我们考虑提取出每种颜色
- 假设这个颜色出现在x和y的位置，如果x和y不构成祖先关系，则DFS序在 $[lx, rx] \times [ly, ry]$ 这个矩形中的所有链都是不可行的



Solution

- 如果二者构成祖先关系，则这个相当于是一个区间补的形式（就是删除一个子树的DFS序，也可以用 $O(1)$ 个矩形表示）



Solution

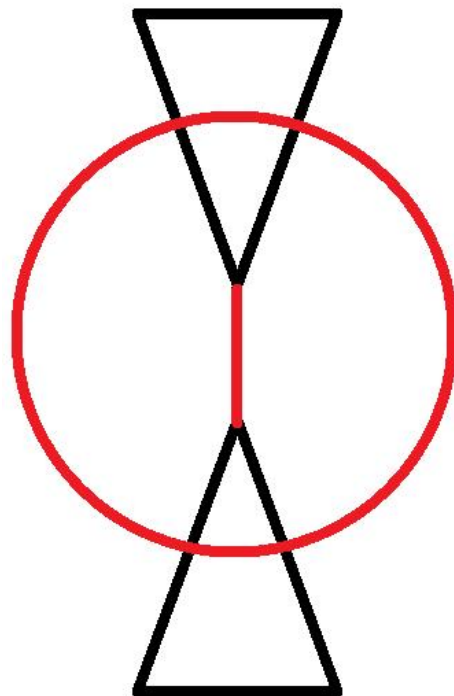
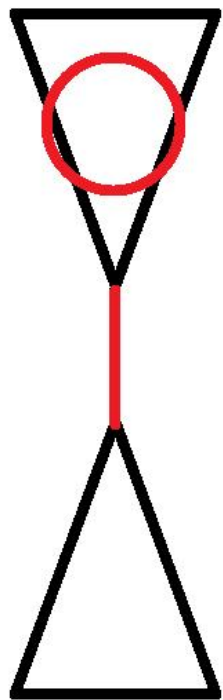
- 所以我们可以预处理出每种颜色所导致的限制条件，然后问题转换为，给定 $O(cn)$ 个矩形，求面积并
- 扫描线维护矩形面积并，总时间复杂度 $O(cn \log n)$

一个有趣的问题

- 给一棵边权为1，有点权的树，每次查询一个距离 a 小于等于 b 的所有点中点权小于 x 的最大的数
- (树邻域前驱)

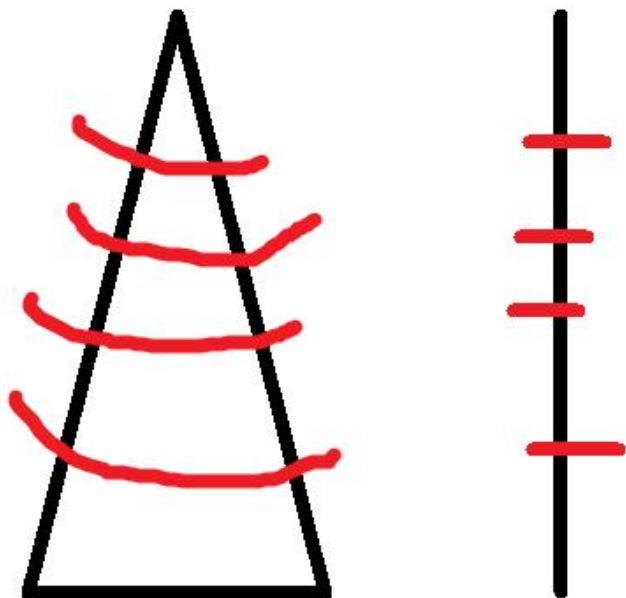
Solution

- 我们把距离 $a \leq b$ 的点集叫做树的 $N(a,b)$ 邻域
- 邻域可以用边分治来划分出来
- 如果邻域在分治中心的一边 |||| 如果邻域被分治中心分成两半



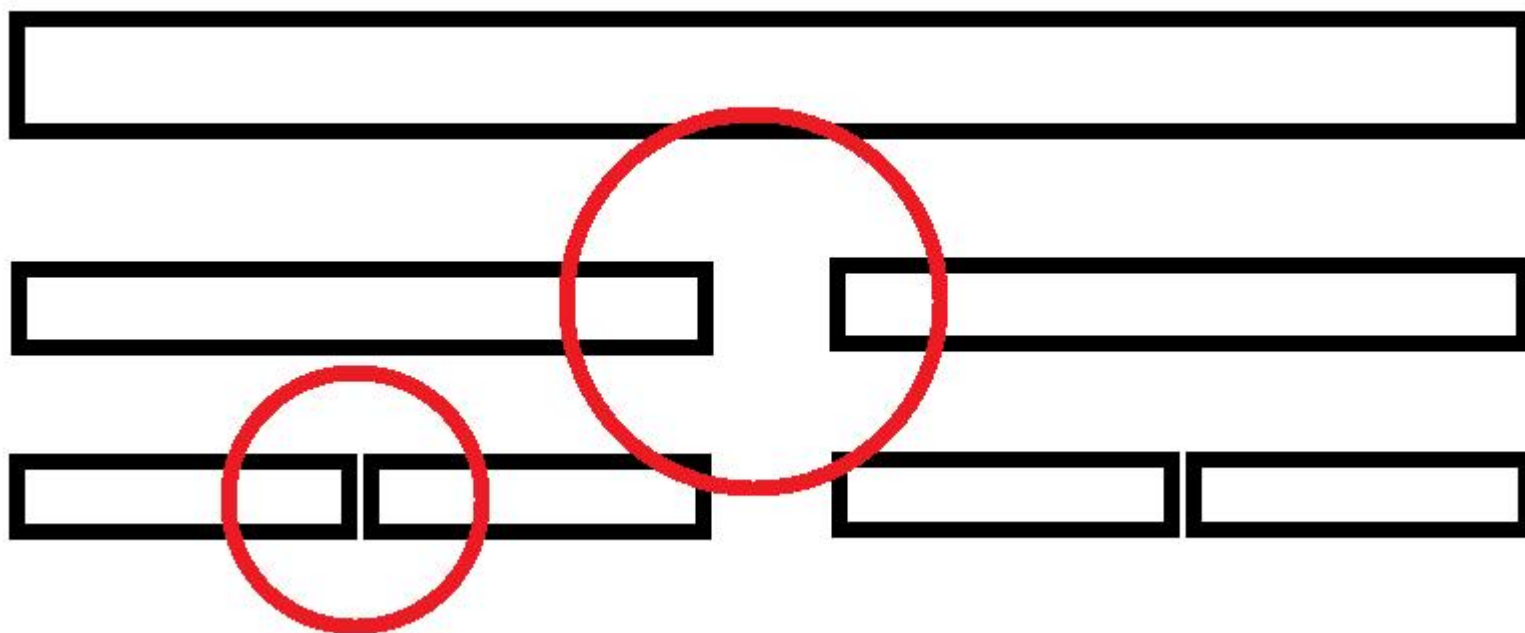
Solution

- 和序列分治类似，我们在第二种情况中，这个询问就是查询两个分治子图的前缀
- 这里分治子图的前缀实际上和树的结构无关了，我们可以将其序列化



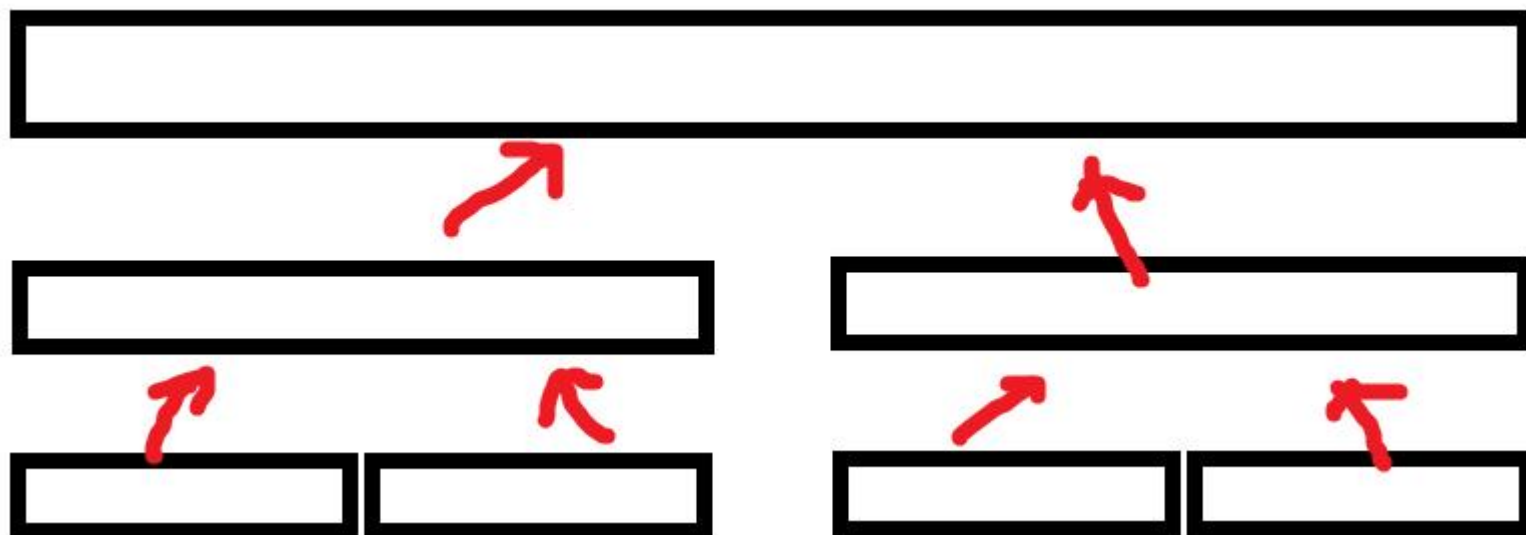
Solution

- 于是可以看做是一棵线段树，每层需要查询一些区间前后缀的前驱：
- 大家记住这个结构，这个是一种重量平衡的静态top tree



Solution

- 如何维护出每个线段树节点的每个前缀和后缀中 x 的前驱呢?
- 可以归并排序上去



Solution

- 在每层中，我们做的问题是：每次查询前缀 $[1, x]$ 中小于 y 的最大的数
- 这个如果我们在线正着做，插入一个数，查询前驱，复杂度 $\Theta(\log \log n)$
- 离线？

Solution

- 每次删除一个数，查询前驱？
- 可以链表维护？
- 然而我们这个问题不太一样，我们有两个集合A和B，每次删除A集合中一个数，或者查询B集合中一个数在A集合中的前驱
- 如果大量B集合中的数都挤在一起，这样是不能用链表维护的

Technology

- 我们用并查集来维护，每次删除一个点之后将其和前面的点合并起来
- 然后我们预处理的时候把B集合每个元素挂在其最近的A集合元素上，这个把AB集合的元素一起排序就行了，可以在之前说的结构上归并上来
- $O(\alpha(n))$

Technology

- 线性并查集：已知合并顺序的并查集可以做到 $O(1)$
- 考虑将并查集分为 $\Theta(\log n)$ 大小的块，这样并查集只用开 $O(n/\log n)$ 大小
- 每一个块里面每个位置是否被删除可以用二进制表示
- 当一个块所有位置都被删除后，将这个块和前面合并
- 我们这样只能定位到是在哪一个块里面，具体定位值需要位运算优化，这个可以做到 $O(1)$

Solution

- 分治并且归并的总复杂度 $O(n \log n)$
- 对于每个询问会拆成2个前缀前驱询问，每个询问是 $O(1)$ 的
- 总时间复杂度 $O((n+m) \log n)$

Luogu 5439 【XR-2】永恒

有一颗 n 个点的永恒的树，树中每个点 $x(1 \leq x \leq n)$ 上都有一个永恒的字符串 $S(x)$ 。

但这世界没有永恒，所有的一切都会随着时光消失。我们只能给每个所谓永恒的东西定义一个永恒值 f 。这个值本身没有意义，只是一个象征罢了。

- 一个字符串 S 的永恒值 $f(S)$ 定义为它的长度 $\text{Len}(S)$ ，即：

$$f(S) = \text{Len}(S)$$

- 树上的一条无向路径 $K = [u, v](u < v)$ 指的是 u, v 之间的简单路径（包括 u, v ），其永恒值 $f(K)$ 定义为路径上所有不同的无序点对 $(x, y)(x \in K, y \in K, x < y)$ 上的字符串 $S(x), S(y)$ 的最长公共前缀 $\text{LCP}(S(x), S(y))$ 的永恒值 $f(\text{LCP}(S(x), S(y)))$ 之和，即：

$$f(K) = \sum_{x \in K, y \in K, x < y} f(\text{LCP}(S(x), S(y)))$$

- 一颗树 T 的永恒值 $f(T)$ 定义为树上所有的无向路径 $[u, v](u \in T, v \in T, u < v)$ 的永恒值之和，即：

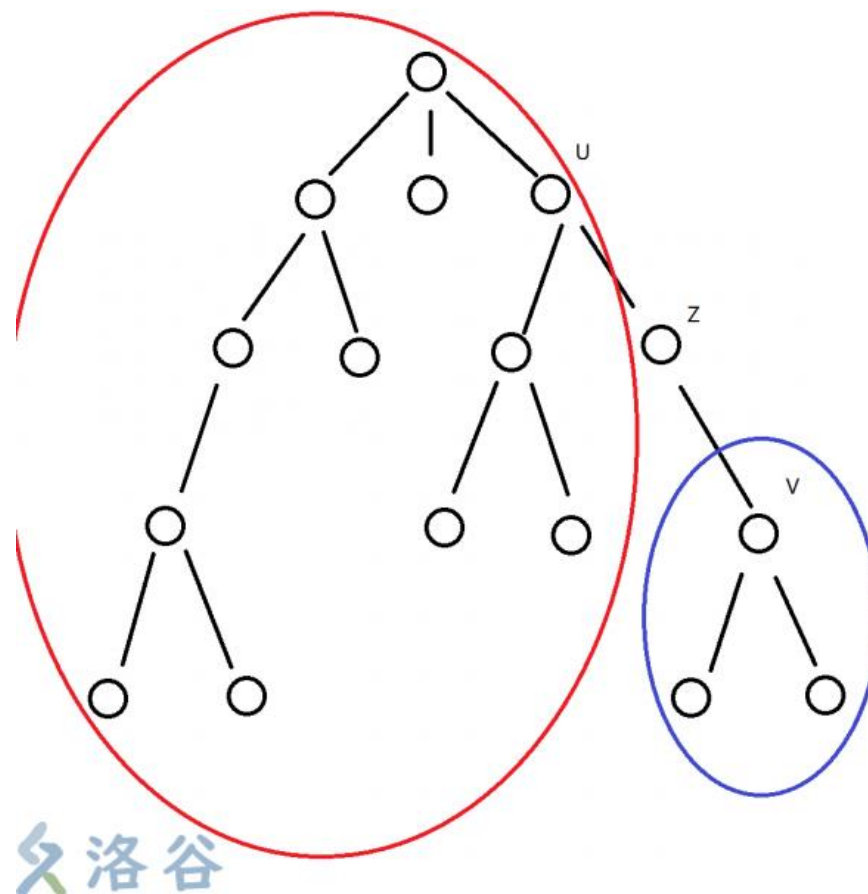
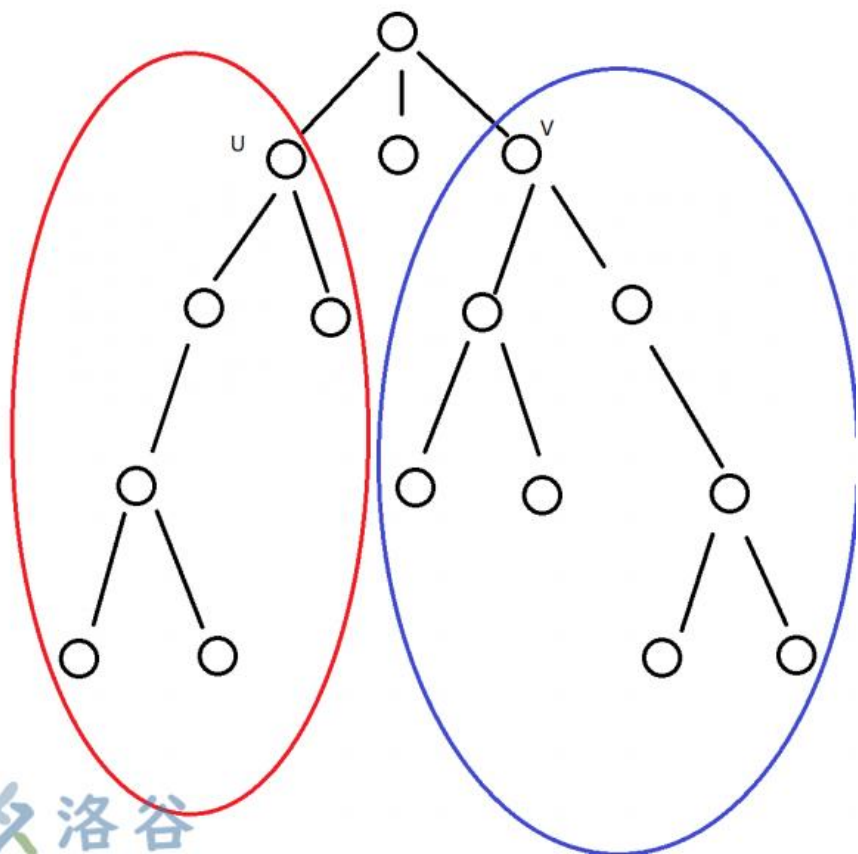
$$f(T) = \sum_{u \in T, v \in T, u < v} f([u, v])$$

特别的是，树中每个点上的字符串都来自一颗永恒的以点 1 为根的 **Trie** 树，即每个树中的点都对应着一个 **Trie** 树中的点，点上的字符串就是 **Trie** 树中从根节点到其对应的点形成的字符串。

你需要求出这棵树的永恒值，答案对 998244353 取模。

Solution

- 首先链的子链可以同个给每个链 $x \rightarrow y$ 加个 x 左边的点个数 $\times y$ 右边的点个数这样的权值，变成普通的链统计



Solution

- 然后树分治一下，由于每个字符串都是trie上的一条根开始的链，发现这个就是对分出来的点集，求两两lca深度和
- 这个显然可以枚举每个点，点到根加点到根和来做掉
- $O(n \log^2 n)$

Solution

- 其实我们只用统计这个信息的话可以虚树维护
- 虚树然后离线计数排序的话可以 $O(n)$ 统计
- 于是这道题可以做到 $O(n \log n)$ 时间

Luogu3292 [SCOI2016]幸运数字

- 一颗带点权的树，求树上两点间异或值最大子集的异或值
- 由于不带修改，所以也算静态问题

Solution

- 链线性基
- 由这道题我们分析一下各种树上分治算法的复杂度吧
- 线性基是很特殊的分治信息，特性：
- 设字节为 w
- 空间 $O(w)$
- 合并 $O(w^2)$
- 插入元素 $O(w)$

Solution1

- HLD: 每次合并 $O(\log^2 n)$ 个信息, 每合并两个信息的代价是 $O(w^2)$, 总复杂度 $O((n+m\log^2 n)w^2)$

Solution2

- LCT: 每次合并 $O(\log n)$ 个信息, 每合并两个信息的复杂度是 $O(w^2)$, 总复杂度 $O((n+m\log n)w^2)$

Solution3

- 倍增：预处理合并 $O(n \log n)$ 个信息，查询每次合并 $O(\log n)$ 个信息，每合并两个信息的复杂度是 $O(w^2)$ ，总复杂度 $O((n+m) \log n w^2)$

Solution4

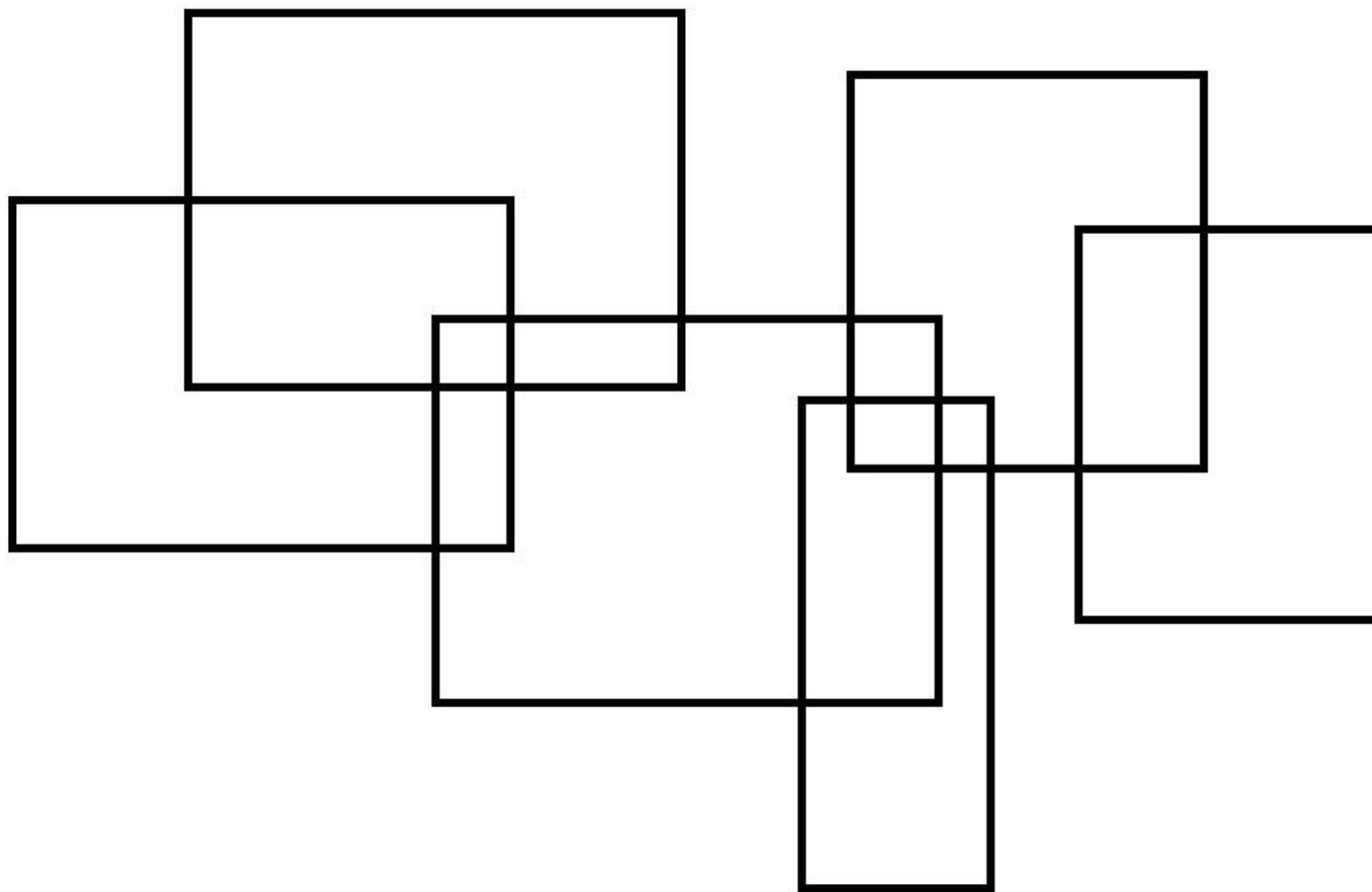
- 树分治：分治过程中总共插入 $O(n \log n)$ 个数，因为树分治是把每条链拆成两条链合并，所以每次查询只需要合并两个信息，总复杂度 $O(n \log n + n^2)$ ，比其他做法优越
- 能不能更低呢？
- 好像有人说可以 $1 \log$ ，但我懂线性基的额外性质所以不知道怎么做

计算几何中的扫描线

- 这类问题看上去更形象一些
- 可以看做是拿着一条线去扫平面

矩形面积并

- 给定二维平面上的堆矩形，求矩形面积的并



Solution

- 先离散化，因为端点个数只有 $O(n)$ 个
- 然后扫描线，使用线段树维护：
- 进入一个矩形的时候区间+1
- 走出一个矩形的时候区间-1
- 这一部分中被覆盖的位置就是全局 >0 的位置
- 这里每个位置加了个权，就是其对应y轴区间长度
- 如何维护呢？

Solution

- 可以发现每个位置都 ≥ 0
- 所以可以用那个维护min和min出现次数的方法维护0位置的个数
- 总复杂度 $O((n+m)\log n)$

Luogu 3268 [JLOI2016]圆的异或并 &BZOJ2758 [SCOI2012]Blinker的噩梦

- 平面上有 n 个多边形（凸包和圆）
- 任意两个多边形 AB 只有两种关系：（1） A 包含 B 或者 B 包含 A ；（2） AB 的公共面积为0。
- 每个多边形有一个值 x 。 m 个查询。
- 分两种：（1）修改某个多边形的值；
- （2）从一点 s 走到另一点 t 。每次走出一个多边形或者进入一个多边形时，都要xor上该多边形的值。输出走到 t 时的值。（由xor的性质和本题定义可得这个值跟走的路径无关）
- ----简洁版题面摘自某博客
- <https://blog.csdn.net/u014609452/article/details/50926290>

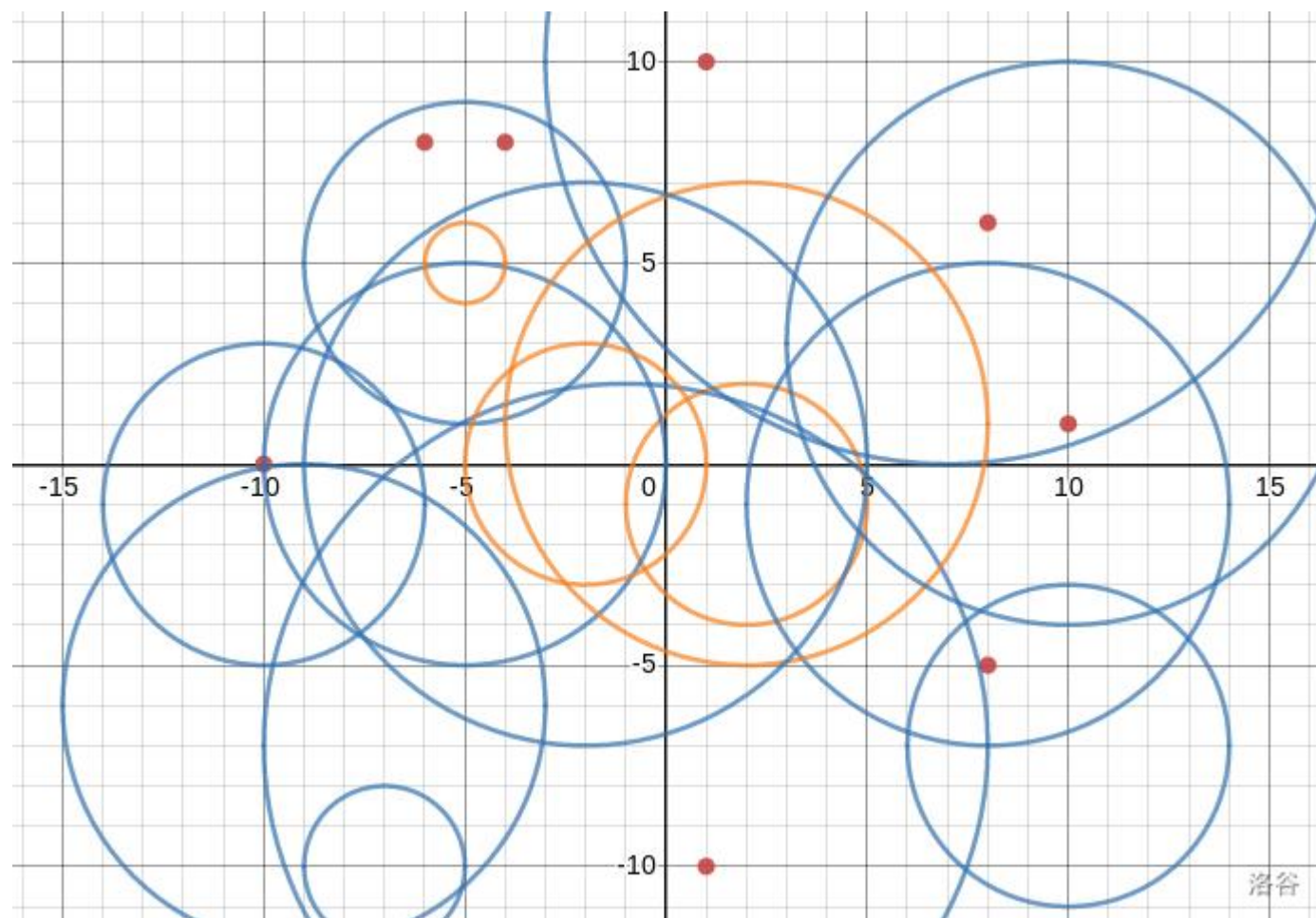
Solution

- 只要是那种给你一些图形，这些图形要么相互包含，要么相离，那就是扫描线把这些图形构成的树建出来，然后再建出来的这个树上面搞的题了
- 可以用平衡树维护扫描线，扫描线上1表示一个多边形的下半部分，-1表示一个多边形的上半部分，类似括号序列的维护方法，每次插入一个图形的时候在平衡树上二分，得到其应该在的位置，二分的时候每个多边形或圆在当前扫描线的位置可以 $O(1)$ 得到
- 没有任何思维难度，但是会很码的无聊东西

Luogu5525 [Ynoi2012]WC2016充满了失望

- 在平面直角坐标系中,
- 给 n 个点, 这 n 个点是可达的, 如果点 A, B 可达则线段 AB 上的点均可达。
- 给 m 个圆, 问有哪些圆满足圆内任意点都是可达的。
- 如果无法理解题意, 可以认为是给了一个凸包, 每次查询一个圆是否被凸包包含

Luogu5525 [Ynoi2012]WC2016充满了失望



Solution

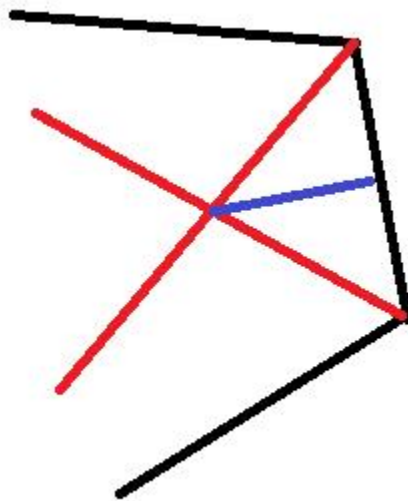
- 可达的点集即为 n 个点的凸包。求出凸包后，只需判断每个圆是否被凸包完全包含。判断半径为 r ，圆心为 O 的圆是否被凸包包含，只需将凸包收缩 r （即将凸包看作半平面交，每个半平面向凸包内部平移距离 r ），判断收缩后的凸包是否包含圆心。

Solution

- 离线，将询问对半径 r 升序排序，将 r 看作时间，模拟凸包收缩的过程。可以使用平衡树 (对于c++，使用std::set是足够的)维护凸包上的每条线段的端点的位置关于 r 的函数，以及凸包上每条线段消失的时间，当一条线段消失时更新相邻两条线段的信息。查询点在凸包内可以像平常一样二分。
- 收缩凸包的时候如何计算出每条线段消失时间呢？

Solution

- 可以发现我们收缩凸包的时候，画出一条线段相邻的两条角平分线，他们的交点位置就是我们这条线段长度变为0时凸包收缩到的位置，于是长度就是垂线长度



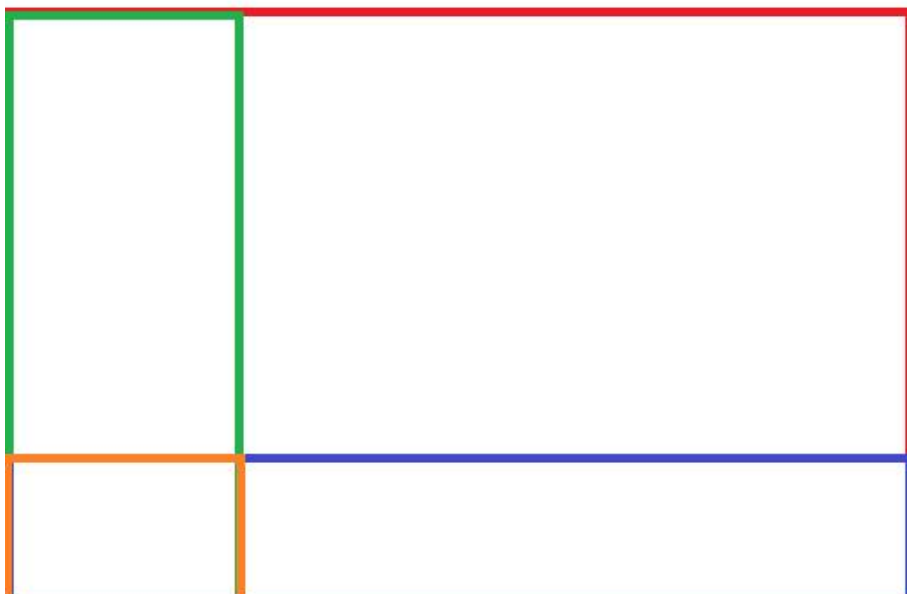
- 总时间复杂度 $O((n+m)\log n)$

Luogu6106 [Ynoi2010]Self Adjusting Top Tree

- 平面上有 n 条线段。
- 共 m 次询问，每次询问给出一个边平行于坐标轴的矩形，问 每条与矩形有交的线段与矩形的交的长度之和 与 所有线段的长度之和 的 比值，要求输出与标准答案的相对误差或绝对误差不超过 $1e-6$ 。

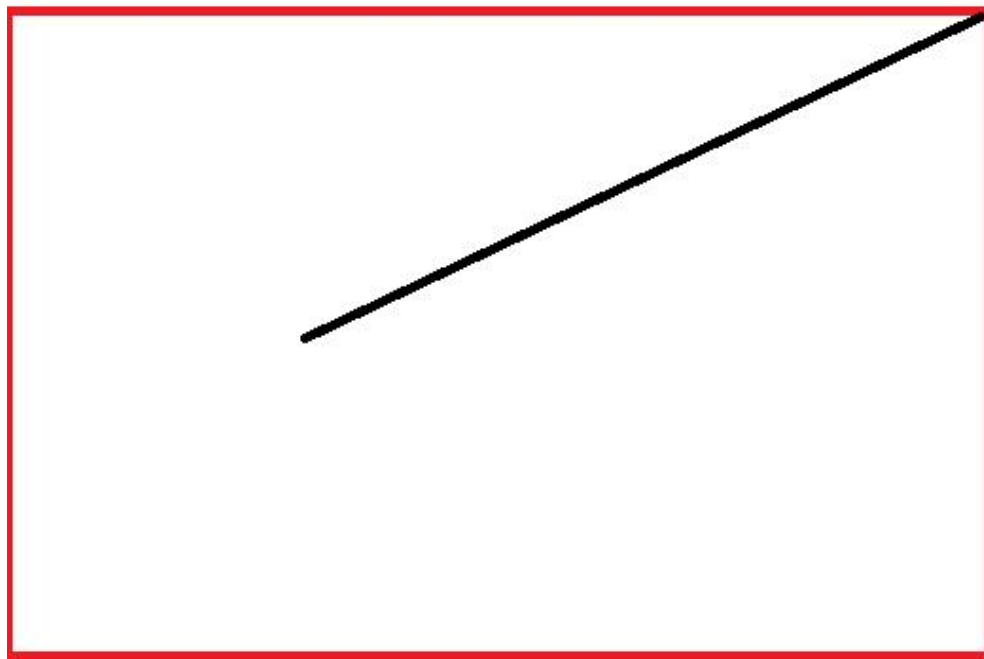
Solution

- 首先考虑所有斜率为正的线段。为负的那些只需要翻转一下坐标系。
- 这是个求和问题,贡献可减,我们把所有的询问容斥成前缀矩形。
- 这里就是通过二维前缀和的差分把1个4-side的矩形差分为4个2-side的矩形



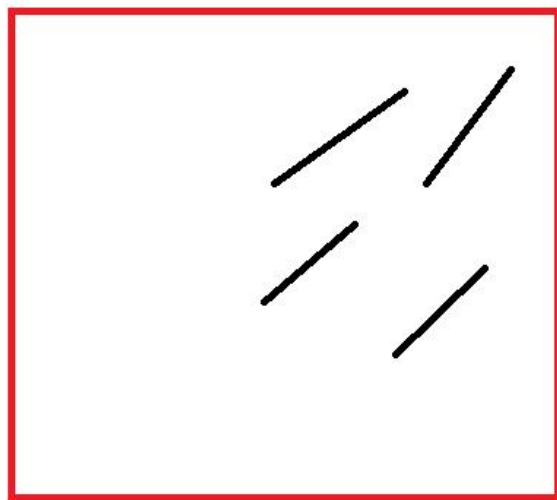
Solution

- 我们发现,此时所有的线段与矩形的位置关系只有四种情况:
- ①相离 ②完全包含 ③与上边界相交 ④与右边界相交
- 这里没有办法同时与上和右边界相交, 除非是刚好在最右上方的点, 这个可以归为④处理



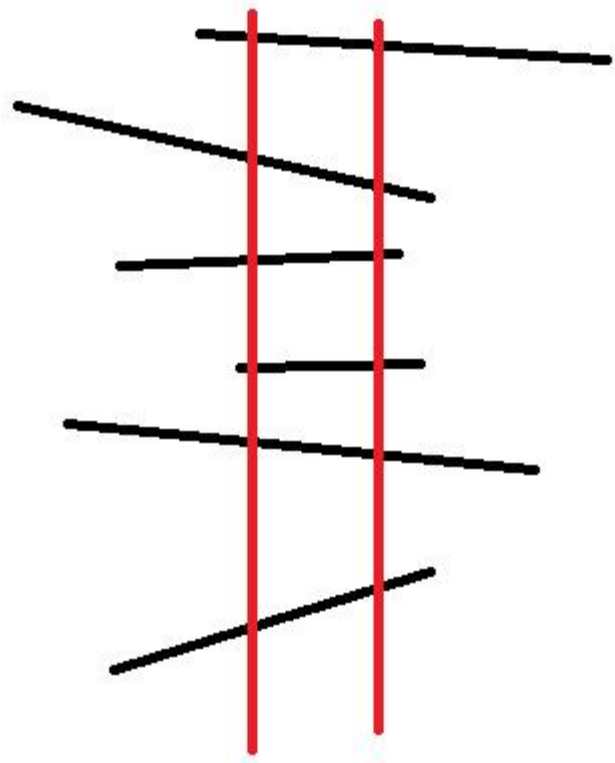
Solution

- 对于完全包含的线段,充要条件是:右上端点在矩形内。这是个经典二维偏序。
- 然后,我们就只需要考虑相交的情况。
- 对于与上边界相交的情况,我们可以翻转坐标系的两轴,或者把这个作为对称情况讨论,这样就只用考虑与右边界相交。



Solution

- 题目给出了一个非常特殊的条件：所有线段不相交。
- 这就表明,如果我们拿一条竖着的直线直线切这些线段,并从左往右移动,所得的交点的顺序是不会改变的。反之则代表线段相交。

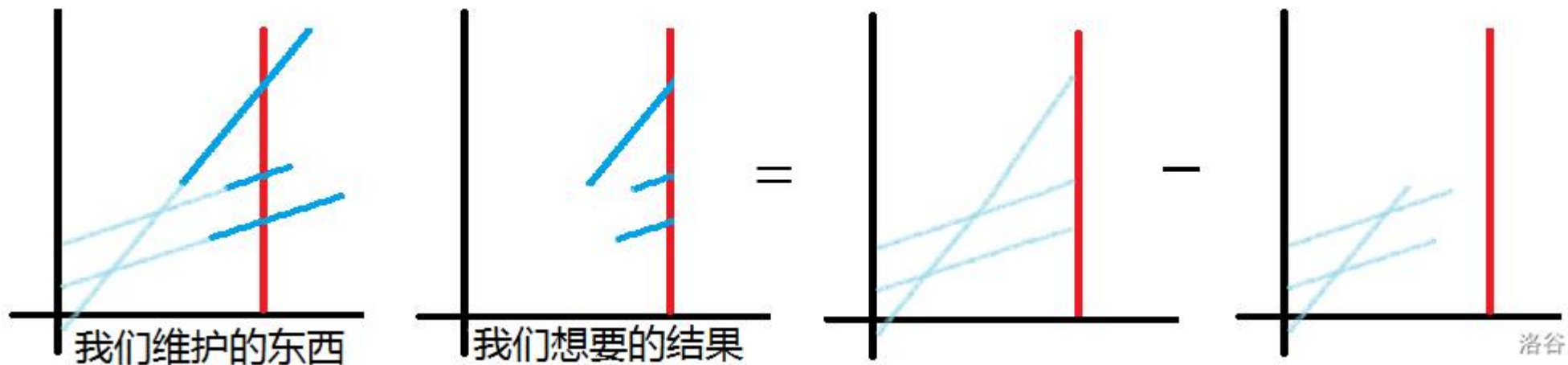


Solution

- 所以,我们可以按照 xx 坐标跑扫描线,用平衡树按照 yy 坐标维护这些线段。
- 每次查询的时候,和矩形右边界相交的线段,在平衡树上一定是一个前缀区间。
- 我们来思考如何统计在右边界左边的总长度。

Solution

- 我们先解出每条线段向右经过一单位所得的长度是多少,称之为单位线长。
- 然后维护单位线长,和从左端点延长到左边界的线段长度和。
- 我们用单位线长的前缀和乘以边界的 x 坐标,就能得到整条线的长度,再减去准备好的延长线长度总和即可。实际上就是在取补集。
- 总时间复杂度 $O((n+m)\log n)$, 常数大概24倍



Thanks for listening